



DWN - Unid I

Exercício 1:

Considere a seguinte configuração de uma aplicação ASP.NET Core:

```
app.UseHttpsRedirection();  
app.UseDefaultFiles();  
app.UseStaticFiles();
```

```
app.MapGet("/api/pets", () =>  
    Results.Ok(new[] { "Mingau", "Luna" }));
```

```
app.Run();
```

O diretório wwwroot contém apenas o arquivo index.html e não contém arquivo correspondente a /api/pets. Considere que as requisições já chegam por HTTPS, de modo que UseHttpsRedirection() não produz redirecionamento. Qual alternativa descreve corretamente o tratamento de / e /api/pets?

A)

UseDefaultFiles() envia diretamente o conteúdo de index.html, enquanto UseStaticFiles() encaminha /api/pets ao endpoint mapeado.

B)

A requisição à raiz somente será atendida depois que o roteamento verificar se existe um endpoint correspondente a /index.html.

C)

UseStaticFiles() procura automaticamente index.html ao receber /, tornando UseDefaultFiles() dispensável nessa configuração.

D)

UseDefaultFiles() reescreve a requisição à raiz para o arquivo padrão; UseStaticFiles() serve esse arquivo; e /api/pets continua sendo atendido pelo endpoint mapeado.

E)

O middleware de arquivos estáticos atende apenas arquivos CSS, JavaScript e imagens; por isso, index.html precisaria ser exposto por MapGet().

Exercício 2:

Uma API criada com Minimal APIs possui os seguintes endpoints:

GET /api/pets/{id:int}

GET /api/pets?especie=Gato&cidade=Curitiba

POST /api/interesses

O endpoint POST /api/interesses recebe, no corpo da requisição, um objeto JSON com PetId, Nome, Email e Mensagem.

De acordo com as convenções de vinculação de parâmetros das Minimal APIs, qual alternativa está correta?

A)

id é obtido da rota; especie e cidade são obtidos da query string; e o objeto enviado ao POST pode ser desserializado do corpo JSON.

B)

id é obtido da query string; especie e cidade são obtidos da rota; e o objeto enviado ao POST é desserializado do corpo JSON.

C)

id é obtido da rota; especie e cidade são obtidos do corpo da requisição; e o objeto enviado ao POST é obtido da query string.

D)

id é obtido da rota; especie e cidade são obtidos dos cabeçalhos; e o objeto enviado ao POST é obtido da query string.

E)

id, especie e cidade são obtidos da query string; e o objeto enviado ao POST precisa ser desserializado manualmente.

Exercício 3:

Uma API mantém a seguinte coleção de animais:

Mingau | Gato | Pequeno | São Paulo

Amora | Gato | Pequeno | Florianópolis

Luna | Cachorro | Médio | Curitiba

Thor | Cachorro | Grande | Belo Horizonte

Os filtros informados na requisição são combinados. Para especie e porte, a comparação ignora diferenças entre letras maiúsculas e minúsculas. Para cidade, a API verifica se o texto informado está contido no nome da cidade, também sem diferenciar maiúsculas de minúsculas.

Considere a requisição:

GET /api/pets?especie=gato&porte=PEQUENO&cidade=paulo

Qual resultado será retornado?

A)

Mingau e Amora, porque ambos atendem aos filtros de especie e porte.

B)

Mingau, Amora e Thor, porque o filtro textual de cidade é aplicado independentemente dos demais critérios.

C)

Apenas Amora, porque o filtro de cidade é aplicado somente quando especie e porte são diferentes.

D)

Nenhum animal, porque o texto informado para especie e porte não possui a mesma combinação de maiúsculas e minúsculas dos dados armazenados.

E)

Apenas Mingau, porque ele atende simultaneamente aos filtros de especie, porte e cidade.

Exercício 4:

Um endpoint executa as validações na seguinte ordem:

```
if (request.PetId <= 0 ||  
    string.IsNullOrEmpty(request.Nome) ||  
    string.IsNullOrEmpty(request.Email) ||  
    string.IsNullOrEmpty(request.Mensagem) ||  
    !request.Email.Contains('@'))  
{  
    return Results.BadRequest();  
}  
  
var pet = pets.FirstOrDefault(p => p.Id == request.PetId);  
  
if (pet is null)  
{  
    return Results.NotFound();  
}  
  
return Results.Ok();
```

Considere duas requisições:

I. PetId = 999, Nome = "Ana",
Email = "ana.exemplo.com", Mensagem = "Tenho interesse"

II. PetId = 999, Nome = "Ana",
Email = "ana@exemplo.com", Mensagem = "Tenho interesse"

O animal de identificador 999 não existe. Quais respostas são produzidas?

A)

I retorna 404; II retorna 400.

B)

I retorna 400; II retorna 404.

C)

Ambas retornam 404, pois a inexistência do animal prevalece sobre a validação.

D)

Ambas retornam 400, pois um identificador inexistente é tratado como entrada inválida.

E)

I retorna 400; II retorna 200, pois os dados da segunda requisição são formalmente válidos.

Exercício 5:

Uma aplicação registra seus endpoints por meio de um método de extensão:

```
app.UseHttpsRedirection();  
app.UseDefaultFiles();  
app.UseStaticFiles();
```

```
app.MapPetEndpoints();
```

```
app.Run();
```

O método é definido assim:

```
public static WebApplication MapPetEndpoints(  
    this WebApplication app)  
{  
    var api = app.MapGroup("/api");  
  
    api.MapGet("/pets", ...);  
    api.MapGet("/pets/{id:int}", ...);  
    api.MapGet("/boas-vindas/{nome}", ...);  
    api.MapPost("/interesses", ...);  
  
    return app;  
}
```

Qual efeito essa organização produz?

A)

O método de extensão registra os endpoints, mas MapGroup("/api") afeta apenas a organização do código e não as URLs.

B)

Os endpoints passam a ser controladores MVC porque foram retirados de Program.cs.

C)

Os endpoints continuam sendo Minimal APIs, e o grupo acrescenta /api aos caminhos registrados nele.

D)

O método de extensão cria uma segunda instância de WebApplication, responsável exclusivamente pelos endpoints agrupados.

E)

O agrupamento mantém as URLs anteriores e adiciona versões alternativas com o prefixo /api.

Exercício 6:

Uma aplicação ASP.NET Core MVC contém:
builder.Services.AddControllersWithViews();

```
var app = builder.Build();
```

```
app.UseHttpsRedirection();  
app.UseStaticFiles();
```

```
app.MapControllerRoute(  
    name: "default",  
    pattern: "{controller}/{action=Index}/{id?}");
```

```
app.Run();
```

Existe um ProdutosController com uma ação pública Index(). O que ocorre ao acessar /Produtos?

A)

Produtos é interpretado como nome da ação, enquanto o controlador recebe o valor padrão Index.

B)

A URL não corresponde à rota, pois id precisa ser informado quando o segmento da ação é omitido.

C)

O middleware de arquivos estáticos procura Produtos.html e transforma a requisição em uma chamada MVC caso o arquivo não exista.

D)

AddControllersWithViews() cria automaticamente uma rota exclusiva para cada controlador, tornando MapControllerRoute() desnecessário.

E)

O primeiro segmento identifica ProdutosController, e a ausência do segundo faz a rota utilizar a ação padrão Index.

Exercício 7:

Um ViewModel é definido assim:

```
public class ProdutoViewModel
{
    [Required]
    [StringLength(60, MinimumLength = 2)]
    public string Nome { get; set; } = "";

    [Range(0, 100)]
    public int Quantidade { get; set; }

    [Required]
    [StringLength(200, MinimumLength = 20)]
    public string Descricao { get; set; } = "";
}
```

Uma ação POST verifica ModelState.IsValid e retorna a mesma view quando o estado é inválido. O formulário envia:

Nome = "A"

Quantidade = 15

Descricao = "Descrição curta"

Qual comportamento é esperado?

A)

Nome e Descricao violam seus comprimentos mínimos, tornando ModelState inválido.

B)

Somente Nome é inválido, pois a primeira falha impede a avaliação das demais propriedades.

C)

O modelo é válido, pois todas as propriedades obrigatórias foram preenchidas e a quantidade está no intervalo permitido.

D)

A requisição é rejeitada pelo roteamento antes da execução da ação porque Data Annotations participam da seleção de endpoints.

E)

O model binder completa automaticamente as strings até os comprimentos mínimos definidos.

Exercício 8:

Um controlador MVC possui:

```
[HttpGet]
```

```
public IActionResult Create()
```

```
{
```

```
    return View(new ProdutoViewModel());
```

```
}
```

```
[HttpPost]
```

```
[ValidateAntiForgeryToken]
```

```
public IActionResult Create(ProdutoViewModel model)
```

```
{
```

```
    if (!ModelState.IsValid)
```

```
        return View(model);
```

```
    Salvar(model);
```

```
    TempData["MensagemSucesso"] = "Produto cadastrado.";
```

```
    return RedirectToAction(nameof(Index));
```

```
}
```

Qual alternativa descreve corretamente as responsabilidades desse fluxo?

A)

A ação GET recebe os dados enviados pelo formulário, enquanto o POST apenas escolhe a próxima view.

B)

TempData precisa ser preenchido antes da validação porque seus valores não permanecem disponíveis após redirecionamentos.

C)

[ValidateAntiForgeryToken] executa as regras de [Required], [Range] e [StringLength], tornando ModelState desnecessário.

D)

O POST recebe os dados associados ao ViewModel, verifica sua validação e, se forem válidos, executa a operação e retorna um redirecionamento.

E)

RedirectToAction() executa a ação Index dentro da mesma resposta HTTP, sem nova requisição do cliente.

Exercício 9:

Uma aplicação registra um filtro MVC com o seguinte código:

```
public void OnActionExecuting(ActionExecutingContext context)
{
    if (context.Controller is Controller controller)
    {
        controller.ViewData["Contexto"] =
            "Área administrativa";
    }
}
```

A mesma informação deve estar disponível nas views de diversas ações. Qual justificativa é mais adequada para o uso desse filtro?

A)

Concentrar no filtro as regras de negócio utilizadas pelas ações, reduzindo a quantidade de serviços da aplicação.

B)

Centralizar uma preocupação transversal executada antes das ações, evitando repetir o mesmo código em diferentes controladores.

C)

Utilizar o filtro como mecanismo principal de roteamento para decidir qual ação será executada.

D)

Manter o conteúdo de ViewData disponível globalmente para todas as requisições futuras da aplicação.

E)

Substituir a validação dos modelos recebidos pelas ações, pois o filtro é executado antes delas.

Exercício 10:

Uma aplicação MVC possui:

Views/

+ - _ViewImports.cshtml

+ - Shared/

| + - _Layout.cshtml

| + - _ItemCard.cshtml

+ - Produtos/

+ - Index.cshtml

+ - Create.cshtml

Em Index.cshtml:

@foreach (var item in Model)

{

<partial name="_ItemCard" model="item" />

}

Em _Layout.cshtml:

<main>

@RenderBody()

</main>

Qual alternativa caracteriza corretamente esses elementos?

A)

O layout executa o controlador da página, a partial substitui a ação e RenderBody() realiza o model binding.

B)

O layout é destinado apenas a referências CSS; a partial cria rotas MVC; e RenderBody() seleciona o controlador.

C)

O layout define a estrutura compartilhada; a partial encapsula um fragmento reutilizável; e `RenderBody()` indica onde o conteúdo da view será renderizado.

D)

Layout e partial representam o mesmo mecanismo, mas partials não podem receber modelos tipados.

E)

A partial concentra regras de negócio reutilizáveis, enquanto o layout executa regras de validação do ViewModel.